

AI-POWERED RESUME SCREENING AND TALENT ACQUISITION

PHASE I REPORT

Submitted by

DANIEL LEVE MANICKAM D A	230701060
DARSHAN M	230701061
DAYANITHI V	230701064
DEEPA S	230701065

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE,

CHENNAI ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY: CHENNAI

BONAFIDE CERTIFICATE

Certified that this Phase I Report titled “**AI-POWERED RESUME SCREENING AND TALENT ACQUISITION**” is the bonafide work of **DANIEL LEVE MANICKAM D A (230701060), DARSHAN M (230701061), DAYANITHI V (230701064), and DEEPA S (230701065)**, who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on these or any other candidates.

SIGNATURE

Dr. J. Manoranjini,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai - 602 105.

SIGNATURE

Dr. J. Jinu Sophia, Ph.D.,
Assistant Professor (SG),
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

High-volume recruitment requires efficient resume search without reducing relevance to exact keyword overlap. This report presents an implemented architecture for PDF ingestion, hybrid retrieval, and cross-encoder reranking, while separating current functionality from proposed extensions. PDF resumes stored in Amazon S3 trigger AWS Lambda and asynchronous Amazon Textract processing. Amazon SNS reports completion to a collection Lambda, which retrieves every paginated result and preserves the extracted JSON with page, block, geometry, and source provenance. A deterministic procedure converts the text into candidate chunks that are embedded with voyage-4-large and stored in MongoDB Atlas. MongoDB Atlas Search performs BM25 lexical retrieval and Atlas Vector Search performs semantic retrieval. Reciprocal Rank Fusion combines both ranked chunk lists, aggregates contributions by candidate, and produces a unique top- K pool. Candidate documents are reconstructed and jointly evaluated with the job description by rerank-2.5. The output contains a semantic relevance score, reranked position, rank movement, and retrieval provenance. In the recorded pilot, BM25 and vector search each returned eight unique candidates, five candidates overlapped, and the hybrid union contained eleven. The reranking output demonstrated controlled reordering of the retrieved pool. Requirement-to-resume evidence alignment, explainable recommendations, alternative-opening matching, and candidate communication are proposed as human-reviewed extensions. The report documents the implementation, mathematical model, pilot observations, evaluation methodology, security considerations, and future development path.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E., F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr.(Mrs.)THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S. N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. MANORANJINI, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, **Dr. J.Jinu Sophia**, Assistant Professor (SG), Department of Computer Science and Engineering, for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr.S.Madhusudhan**, Associate Professor, Department of Computer Science and Engineering, for his useful tips during our reviews to build our project.

DANIEL LEVE MANICKAM D A - 230701060

DARSHAN M - 230701061

DAYANITHI V - 230701064

DEEPA S - 230701065

TABLE OF CONTENTS

ABSTRACT	i
ACKNOWLEDGEMENT	ii
LIST OF TABLES	vi
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS	viii
1 INTRODUCTION	1
1.1 GENERAL	1
1.2 BACKGROUND AND NEED	2
1.3 PROBLEM STATEMENT	2
1.4 OBJECTIVES	3
1.5 EXISTING SYSTEM	4
1.6 PROPOSED SYSTEM	5
1.7 SCOPE AND BOUNDARIES	5
1.8 ORGANISATION OF THE REPORT	6
2 LITERATURE REVIEW	8
2.1 GENERAL	8
2.2 RESUME PARSING AND DOCUMENT INTELLIGENCE	8
2.3 SKILL EXTRACTION AND NORMALISATION	9
2.4 LEXICAL, DENSE, AND HYBRID RETRIEVAL.....	10
2.5 CROSS-ENCODER RERANKING	10
2.6 PERSON-JOB FIT AND MATCHING	11
2.7 EXPLAINABLE AI IN RECRUITMENT.....	12
2.8 FAIRNESS, PRIVACY, AND GOVERNANCE.....	12
2.9 RESEARCH GAP AND SUMMARY	13

3	SYSTEM ANALYSIS AND DESIGN	15
3.1	DESIGN PRINCIPLES.....	15
3.2	FUNCTIONAL REQUIREMENTS.....	16
3.3	NON-FUNCTIONAL REQUIREMENTS	17
3.4	OVERALL SYSTEM ARCHITECTURE	18
3.5	IMPLEMENTED PROCESS MODEL.....	19
3.6	DATA DESIGN.....	20
3.7	MODULE INTERFACES	21
3.8	DEPLOYMENT AND SECURITY DESIGN	22
4	IMPLEMENTED METHODOLOGY AND PROPOSED EXTENSIONS	23
4.1	END-TO-END METHOD	23
4.2	RESUME INGESTION AND VALIDATION	23
4.3	DOCUMENT EXTRACTION AND PREPROCESSING.....	24
4.4	JOB-DESCRIPTION QUERY MODELLING.....	25
4.5	EMBEDDING AND VECTOR INDEXING	26
4.6	HYBRID CANDIDATE RETRIEVAL	26
4.7	CROSS-ENCODER RERANKING.....	27
4.8	COMPUTATIONAL AND STORAGE MODELS	28
4.9	PROPOSED EVIDENCE ALIGNMENT	29
4.10	PROPOSED ALTERNATIVE-OPENING MATCHING.....	30
4.11	PROPOSED CANDIDATE COMMUNICATION.....	30
4.12	OUTPUT, AUDITABILITY, AND OBSERVABILITY	31
5	IMPLEMENTATION, PILOT RESULTS, AND VALIDATION PLAN	32
5.1	IMPLEMENTED COMPONENTS.....	32
5.2	TEST STRATEGY.....	32
5.3	EVALUATION METRICS	33
5.4	OBSERVED PILOT RETRIEVAL	35
5.5	OBSERVED RERANK MOVEMENT	35

5.6	EXPERIMENTAL PROTOCOL	36
5.7	RISK, ETHICS, AND MITIGATION.....	37
5.8	ACHIEVEMENTS AND SUCCESS CRITERIA	37
6	CONCLUSION AND FUTURE WORK	39
6.1	CONCLUSION	39
6.2	FUTURE WORK.....	39
	REFERENCES	41

LIST OF TABLES

1.1	Report organisation and principal outputs	7
2.1	Synthesis of reviewed approaches and adopted response . . .	14
3.1	Principal functional requirements	16
3.2	Non-functional requirements.....	17
3.3	Core data entities	20
4.1	Proposed explanation-record fields.....	29
5.1	Planned evaluation matrix	34

LIST OF FIGURES

3.1	Implemented hybrid retrieval architecture with proposed downstream extensions.....	18
3.2	Implemented ingestion, retrieval, and review interactions	19
3.3	Logical deployment view	22
4.1	Hybrid retrieval and cross-encoder reranking method.....	24
5.1	Unique-candidate coverage observed for one pilot job description	35
5.2	Retrieval-rank to reranking-rank movement in the recorded pilot output	36

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ANN	Approximate Nearest Neighbour
API	Application Programming Interface
ATS	Applicant Tracking System
BFF	Backend for Frontend
CI/CD	Continuous Integration and Continuous Delivery
DPR	Dense Passage Retrieval
GDPR	General Data Protection Regulation
IAM	Identity and Access Management
JD	Job Description
LLM	Large Language Model
ML	Machine Learning
NER	Named Entity Recognition
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
PDF	Portable Document Format
PII	Personally Identifiable Information
REST	Representational State Transfer
S3	Amazon Simple Storage Service
SBERT	Sentence-BERT
SHAP	SHapley Additive exPlanations
BM25	Best Matching 25 lexical ranking function
RRF	Reciprocal Rank Fusion
SNS	Amazon Simple Notification Service
UI	User Interface
XAI	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Recruitment screening has become a high-volume information retrieval problem. A single technical vacancy can attract hundreds of applications, while campus and graduate programmes may receive several thousand resumes within a short period. Recruiters must compare these documents against detailed role requirements while also meeting operational deadlines. Manual review remains essential for judgement, but unaided review is difficult to apply consistently at this scale. Fatigue, time pressure, and differences in reviewer interpretation can cause qualified candidates to be missed.

Applicant tracking systems reduce administrative workload through document storage, search, and workflow management. However, many screening mechanisms still depend heavily on exact keywords, Boolean rules, or manually assigned filters. A resume describing “development of REST interfaces using FastAPI” may demonstrate the same capability as a requirement for “backend API engineering,” even though the phrases share limited vocabulary. Lexical methods can treat these descriptions as unrelated and rank the candidate poorly. Semantic representations address this weakness by comparing meaning rather than exact wording.

The proposed work treats screening as decision support rather than automated hiring. The system narrows a large corpus, refines relevance over a manageable candidate pool, and presents evidence for recruiter review. Final decisions remain with authorised recruiters. This division of responsibility supports scale without removing accountability.

1.2 BACKGROUND AND NEED

Modern resumes are heterogeneous PDF documents containing free text, lists, tables, columns, icons, and inconsistent headings. Before retrieval, the platform must recover text together with page and layout provenance. Amazon Textract therefore supplies document text and blocks, while the application preserves the raw response rather than claiming that the service has already produced verified semantic profile fields.

Candidate generation combines two complementary signals. BM25 lexical retrieval preserves exact technologies, certifications, and role terminology. Vector semantic retrieval uses chunk embeddings to recover related meaning expressed with different wording. Reciprocal Rank Fusion merges both ranked chunk lists without normalising incompatible raw scores, after which duplicate chunks are consolidated into unique candidate identifiers. A cross-encoder then performs deeper job–resume comparison only over this candidate pool.

Recruitment also requires explanation, privacy, and auditability. A score alone does not show whether the system found evidence of a mandatory qualification or merely detected broad topical similarity. The proposed evidence record therefore links every supported requirement to a resume passage and records model versions, timestamps, and audit references.

1.3 PROBLEM STATEMENT

Existing resume-screening workflows exhibit four connected limitations. First, exact-word matching cannot reliably recognise semantically equivalent skills and experiences. Second, applying a deep pairwise model to an entire resume corpus creates unacceptable computational cost and latency. Third, opaque ranking scores do not provide recruiters or candidates with verifiable reasons. Fourth, candidate information is sensitive personal data and requires controlled access, retention, and audit mechanisms.

The problem addressed in this project is therefore: *how can a recruitment platform combine exact-term and semantic retrieval, efficiently rerank a focused candidate pool, and preserve sufficient provenance for responsible human review?* The current solution targets PDF documents, chunk-level retrieval, candidate deduplication, reproducible configuration, and auditable rank movement. Evidence-grounded explanation and candidate communication extend this foundation in future work.

The project does not attempt to predict long-term employee performance, infer personality, analyse video interviews, perform background verification, or autonomously reject applicants. These exclusions are deliberate. Such tasks require different evidence, introduce higher ethical risk, and extend beyond the objective of evidence-grounded screening.

1.4 OBJECTIVES

The primary objective is to design and implement an AI-assisted talent acquisition platform that separates scalable candidate retrieval from precise relevance assessment. The supporting objectives are:

1. to ingest PDF resumes through an event-driven S3, Lambda, Textract, and SNS workflow;
2. to preserve text, blocks, pages, bounding boxes, source keys, and processing metadata;
3. to generate deterministic section-aware candidate chunks with page provenance and content hashes;
4. to embed candidate chunks and job descriptions with the configured voyage-4-large model;
5. to combine BM25 lexical retrieval and vector semantic retrieval through RRF;

6. to aggregate chunk contributions and deduplicate the result into a top- K candidate pool;
7. to reconstruct candidate documents and rerank them with the rerank-2.5 cross-encoder;
8. to preserve retrieval ranks, matched chunks, source references, and rank movement; and
9. to extend the verified pipeline with evidence alignment and human-reviewed communication.

These objectives define both the functional system and the conditions under which its outputs can be considered useful. A high ranking is insufficient if the supporting evidence cannot be inspected, reproduced, and challenged.

1.5 EXISTING SYSTEM

Conventional screening systems commonly combine manual review, keyword searches, Boolean filters, and rules for education or years of experience. They are operationally simple and provide predictable filtering, but they inherit the vocabulary of the rule author. Synonyms, abbreviations, related tools, and evidence expressed through project descriptions can be overlooked. Term-weighting methods such as TF-IDF reduce the influence of common words but remain dependent on lexical overlap.

End-to-end neural matching systems improve semantic sensitivity, yet a single model applied to every resume can be difficult to scale and audit. Historical hiring labels may also reproduce past organisational preferences. Post-hoc explanation methods can approximate feature influence, but an approximation of model behaviour is not the same as a traceable statement linking a requirement to a specific resume passage.

The practical gap is not the absence of ranking algorithms; it is the difficulty of combining reliable extraction, exact-term search, semantic search, candidate-level fusion, focused reranking, and provenance within one reproducible workflow. The implemented system addresses this retrieval path and provides a stable interface for later explanation and communication services.

1.6 PROPOSED SYSTEM

The implemented platform begins with PDF ingestion and asynchronous document extraction. Successful Textract output is converted into deterministic candidate chunks and embedded with voyage-4-large. MongoDB Atlas executes BM25 lexical search and vector semantic search over those chunks. RRF aggregates contributions from both ranked lists and returns unique candidates. The reranking worker reconstructs each candidate from all stored chunks and applies rerank-2.5 against the job description. Proposed extensions align requirements with evidence and use reviewed evidence for explanations and communication.

The architecture separates components with different performance profiles. Full-corpus embedding and search prioritise throughput. Pairwise reranking prioritises precision on a small subset. Explanation and reporting operate on structured records rather than raw model prose. This separation allows each component to be tested, replaced, or versioned independently.

1.7 SCOPE AND BOUNDARIES

The verified scope includes PDF upload, event-driven Textract extraction, raw JSON preservation, deterministic chunking, voyage-4-large embeddings, MongoDB Atlas Search, MongoDB Atlas Vector Search, RRF candidate aggregation, candidate reconstruction, rerank-2.5 scoring, rank movement, and retrieval provenance. Requirement-level evidence alignment, explanation generation, alternative-opening matching, and candidate communication are proposed extensions.

Out-of-scope capabilities include full integration with commercial applicant tracking suites, automatic interview scheduling, video or voice analysis, personality inference, background verification, payroll processing, and autonomous employment decisions. The platform does not infer protected attributes and does not use names, photographs, or demographic characteristics as ranking features. Recruiters remain responsible for confirming evidence and deciding the next action.

The current deliverable is an integrated ingestion, retrieval, and reranking implementation together with architecture, data contracts, pilot outputs, risk analysis, and evaluation protocol. Subsequent development will implement evidence services and conduct labelled multi-query evaluation. This boundary prevents future capabilities from being presented as current functionality.

1.8 ORGANISATION OF THE REPORT

Chapter 1 introduces the problem, objectives, scope, and proposed solution. Chapter 2 reviews work on document intelligence, skill extraction, dense retrieval, reranking, person–job fit, explainability, fairness, privacy, and skill-gap feedback. Chapter 3 converts these findings into system requirements and presents the logical, data, deployment, interface, and security designs.

Chapter 4 develops the implemented methodology in detail, following the resume from PDF upload through extraction, chunking, hybrid retrieval, fusion, and reranking, followed by clearly separated proposed extensions. Chapter 5 presents implementation status, pilot observations, test strategy, evaluation metrics, experiment design, risks, and success criteria. Chapter 6 concludes the report and identifies future work. References follow the prescribed university format.

Table 1.1: Report organisation and principal outputs

Chapter	Principal output
1	Problem definition, objectives, scope, and contribution
2	Research synthesis and identified technical gaps
3	Requirements, architecture, data, deployment, and controls
4	Module-level methodology and end-to-end processing flow
5	Implementation, pilot observations, and validation protocol
6	Conclusions and future work plan

CHAPTER 2

LITERATURE REVIEW

2.1 GENERAL

The literature review is organised around the responsibilities that a production-oriented screening platform must satisfy. Studies are considered not only for predictive performance but also for document robustness, scalability, evidence traceability, dependence on historical labels, and applicability to human-supervised recruitment. The review draws on transformer representation learning, occupational skill extraction, information retrieval, person–job fit modelling, explainable AI, fairness research, privacy regulation, and recommender systems.

No single reviewed approach covers the complete project scope. Resume parsers may recover structure but do not rank. Dense retrievers scale but do not model every token-level interaction. Cross-encoders refine relevance but are expensive over full corpora. Post-hoc explainers estimate feature influence without necessarily identifying a verifiable source passage. The system architecture therefore integrates complementary methods while defining strict interfaces between them.

2.2 RESUME PARSING AND DOCUMENT INTELLIGENCE

Early resume parsers relied on fixed templates, regular expressions, and hand-written section rules. These methods work on familiar layouts but transfer poorly to multi-column designs, tables, scanned pages, and non-standard headings. Transformer-based named-entity recognition improves extraction of skills, organisations, roles, and dates by using context. End-to-end systems have also combined parsing and job matching, demonstrating the value of structured resume representations.

Cloud document-intelligence services provide asynchronous OCR and structure extraction for heterogeneous documents. They identify lines, tables, key–value relationships, and reading order. For the proposed system, extraction confidence and source coordinates must be retained because later explanation depends on tracing a claim to the original resume. Document extraction is thus treated as a provenance-producing stage rather than a text conversion utility.

The remaining challenge is error propagation. Incorrect reading order or section classification can alter embeddings and evidence alignment. The design responds with validation states, correction support, and explicit extraction-confidence metadata.

2.3 SKILL EXTRACTION AND NORMALISATION

Skill extraction is complicated by abbreviations, product versions, aliases, and the difference between mentioning a tool and demonstrating its use. SkillSpan shows that domain-adapted language models improve identification of skill spans in job-market text. JobBERT uses skill co-occurrence to support job-title understanding, while ESCO provides a multilingual occupational vocabulary for normalisation.

Taxonomies improve consistency but cannot be treated as complete. Software tooling evolves faster than curated vocabularies, and employers use organisation-specific terminology. The proposed design combines an alias lexicon with semantic similarity and preserves the original phrase. Normalisation therefore adds a canonical representation without erasing source evidence.

A further distinction is made between claimed skills and evidenced competence. A skills list may state “Python,” whereas an experience statement may describe building a FastAPI service. Both are retained, but evidence location and context influence the strength assigned during requirement alignment.

2.4 LEXICAL, DENSE, AND HYBRID RETRIEVAL

Lexical retrieval and dense retrieval preserve different signals. BM25 rewards exact term evidence while correcting for document length and corpus frequency. It is effective when a job description names a certification, product, programming language, or role explicitly. Dense retrieval maps queries and candidate chunks into a shared vector space and can recover conceptually related wording even when the surface vocabulary differs. Neither signal is uniformly sufficient for resume screening.

The implemented system therefore runs MongoDB Atlas Search BM25 and MongoDB Atlas Vector Search over the same chunk collection. The vector branch uses 1024-dimensional representations produced by voyage-4-large. Ranked chunk results from both branches are fused with Reciprocal Rank Fusion (RRF). RRF uses rank positions instead of attempting to calibrate incomparable raw score scales, and evidence from multiple matching chunks can accumulate for the same candidate before candidate identifiers are deduplicated.

This hybrid design treats retrieval as candidate generation. Its goal is to create a diverse, high-recall top- K pool for the more expensive cross-encoder. The pool size remains configurable and should be calibrated using Recall@ K , hybrid coverage, latency, and reranking workload on governed evaluation data.

2.5 CROSS-ENCODER RERANKING

Cross-encoders process the job description and candidate text jointly. Full attention between both inputs allows the model to recognise fine-grained interactions, such as whether a tool was used in a relevant context or merely named. Passage reranking research demonstrates consistent precision improvements when a cross-encoder is applied to a smaller set returned by a first-stage retriever.

The method is not suitable for exhaustive corpus scoring because each job–resume pair requires joint evaluation. Hybrid retrieval confines this cost to the top- K pool. The current implementation reconstructs one candidate document from the stored chunks and submits the job description and reconstructed document to the configured rerank-2.5 model. Its output is treated as a semantic relevance score, not as a hiring probability or percentage match.

Model versioning is important because a revised reranker can change candidate order. Every screening run therefore records the model identifier, preprocessing version, query, retrieved set, and timestamp so that the ranking can be reproduced and audited.

2.6 PERSON–JOB FIT AND MATCHING

Person–job fit research has used convolutional encoders, hierarchical attention, graph representations, and transformers to learn relationships between candidate profiles and job requirements. These systems demonstrate that requirement structure matters: education, experience, skills, and responsibilities should not be collapsed into an undifferentiated document score.

Many supervised approaches depend on historical hiring outcomes. Such labels may be sparse for a new deployment and may reflect prior organisational preferences. The proposed project avoids treating historical selection as unquestioned ground truth. Recruiter judgements used for evaluation will be collected with explicit criteria, and the system will expose requirement-level evidence rather than learn directly from final hiring decisions alone.

The current architecture deliberately limits its implemented output to retrieval and semantic reranking. Requirement-level policy checks may later be added as a separate, reviewable layer; they are not inferred from the semantic relevance score and are not presented as implemented in this report.

2.7 EXPLAINABLE AI IN RECRUITMENT

LIME and SHAP estimate how input features influence a model prediction and are widely used to explain opaque systems. Reviews of explainable recruitment systems note their popularity but also emphasise that local approximations may not reveal the model’s actual internal reasoning. In recruitment, an explanation must be useful to a reviewer and safe to communicate to a candidate.

The proposed extension adopts evidence alignment instead of unrestricted explanation generation. Each job requirement would be linked to one or more resume spans with source location, confidence, and support state. A natural-language component may later render these records, but it must not introduce new facts. This design would make explanations inspectable while reusing provenance already preserved by extraction and chunking.

When this extension is implemented, faithfulness can be evaluated by checking whether every explanation claim maps to a stored evidence entry. This is stronger than measuring whether an explanation merely sounds plausible.

2.8 FAIRNESS, PRIVACY, AND GOVERNANCE

Research has documented disparate behaviour in automated recruitment systems and warns that historical labels can encode past inequities. Fairness cannot be inferred from the absence of protected attributes because proxies may remain in institution names, locations, career gaps, or language style. Audit work further shows that a system may appear fair simply because it performs little substantive evaluation.

Privacy requirements are equally important. Resumes contain identity, contact, education, employment, and sometimes sensitive personal information. The design applies least-privilege access, encryption, retention controls, purpose limitation, audit logs, and deletion workflows. Identity attributes are separated from ranking features wherever practical.

Human oversight is operationalised through review screens, evidence links, correction mechanisms, and explicit final-decision ownership. Governance is thus implemented as system behaviour rather than presented only as a policy statement.

2.9 RESEARCH GAP AND SUMMARY

The reviewed literature provides strong components but leaves an integration gap. Resume parsing work focuses on extraction accuracy; retrieval work focuses on search quality; reranking work focuses on ordering; explainability work often uses post-hoc attribution; and skill-gap systems commonly operate independently of actual screening evidence. Few systems connect these elements through a single auditable record.

The implemented contribution is a modular architecture in which document provenance flows from asynchronous extraction to deterministic chunks, hybrid retrieval, candidate reconstruction, and cross-encoder reranking. BM25 protects exact terminology, vector search addresses vocabulary variation, and RRF combines both ranked signals without raw-score calibration. Evidence alignment, alternative-opening matching, and candidate communication remain proposed extensions built on the same provenance foundation.

Table 2.1: Synthesis of reviewed approaches and adopted response

Area	Recurring limitation	Project response
Extraction	Layout sensitivity and lost provenance	Textextract JSON with page, block, and geometry data
Lexical retrieval	Vocabulary-sensitive matching	BM25 exact-term evidence
Dense retrieval	Possible exact-term misses	Vector semantic retrieval over chunks
Score fusion	Incomparable raw score scales	Rank-based RRF with candidate aggregation
Reranking	High full-corpus cost	Cross-encoder evaluation of the top- K pool
Explanation	Plausible but unverifiable prose	Proposed requirement-to-evidence records

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 DESIGN PRINCIPLES

The system is guided by seven principles: complementary retrieval, scalability, traceability, reproducibility, privacy, modularity, and human control. Complementary retrieval retains both exact lexical evidence and semantic similarity. Scalability is achieved by indexing resume chunks and limiting expensive pairwise scoring to a retrieved candidate pool. Traceability requires every chunk to retain its document and page origin. Reproducibility requires frozen configuration and model metadata for each run.

Privacy is supported through private object storage, least-privilege service roles, controlled secrets, and limited propagation of raw resume text. Modularity allows extraction, chunking, embedding, lexical search, vector search, fusion, and reranking to be tested independently. Human control ensures that the ranked output supports recruiter review rather than representing an autonomous employment decision.

These principles become acceptance criteria. A component is not considered complete solely because it produces output; it must also preserve the metadata required by downstream review.

3.2 FUNCTIONAL REQUIREMENTS

The verified requirements describe the complete implemented path from document arrival to the reranked output. Each requirement corresponds to a repository module, service event, stored artefact, or observable result; proposed downstream functions are identified separately.

Table 3.1: Principal functional requirements

ID	Requirement
FR-01	The system shall accept PDF resumes in a private Amazon S3 input location.
FR-02	An S3 object-created event shall initiate asynchronous Amazon Textract processing through AWS Lambda.
FR-03	The callback processor shall paginate GetDocumentTextDetection using NextToken and preserve raw block-level JSON.
FR-04	The system shall write extracted output to extracted/<job-id>.json.
FR-05	Extracted text shall be divided into deterministic candidate chunks with source provenance.
FR-06	Each candidate chunk shall receive a 1024-dimensional voyage-4-large embedding and be stored in MongoDB Atlas.
FR-07	A job description shall be evaluated through BM25 lexical retrieval and vector semantic retrieval.
FR-08	Ranked chunk results shall be combined with RRF using $k = 60$ and accumulated at candidate level.
FR-09	Duplicate candidate identifiers shall be removed before producing the configurable top- K candidate pool.
FR-10	Candidate documents shall be reconstructed from stored chunks for cross-encoder evaluation.
FR-11	The configured rerank-2.5 model shall produce semantic relevance scores and a reranked list.
FR-12	Output shall preserve retrieval provenance and record movement from retrieval rank to reranking rank.
FR-13	Future evidence and communication functions shall consume reviewed outputs without altering the implemented ranking record.

3.3 NON-FUNCTIONAL REQUIREMENTS

Table 3.2: Non-functional requirements

Category	Requirement
Performance	Upload and extraction shall be asynchronous; search latency and reranking latency shall be measured separately.
Scalability	Storage, extraction workers, vector search, and reranking workers shall scale independently.
Reliability	Processing shall be idempotent, retryable, and observable; failed jobs shall enter a review queue.
Security	Data shall use private storage, encrypted transport, least-privilege IAM, and protected external-service credentials.
Privacy	Raw resumes and derived text shall be retained only for the authorised processing purpose and governed by deletion controls.
Explainability	Proposed requirement assessments shall link to stored evidence or state that no evidence was located.
Reproducibility	Model identifiers, search parameters, source hashes, and timestamps shall be recorded.
Usability	Reranked results shall retain candidate identifiers, scores, provenance, and rank movement in readable output.
Maintainability	Components shall expose versioned interfaces and automated tests.
Portability	Cloud resources shall be provisioned through infrastructure-as-code where practical.

3.4 OVERALL SYSTEM ARCHITECTURE

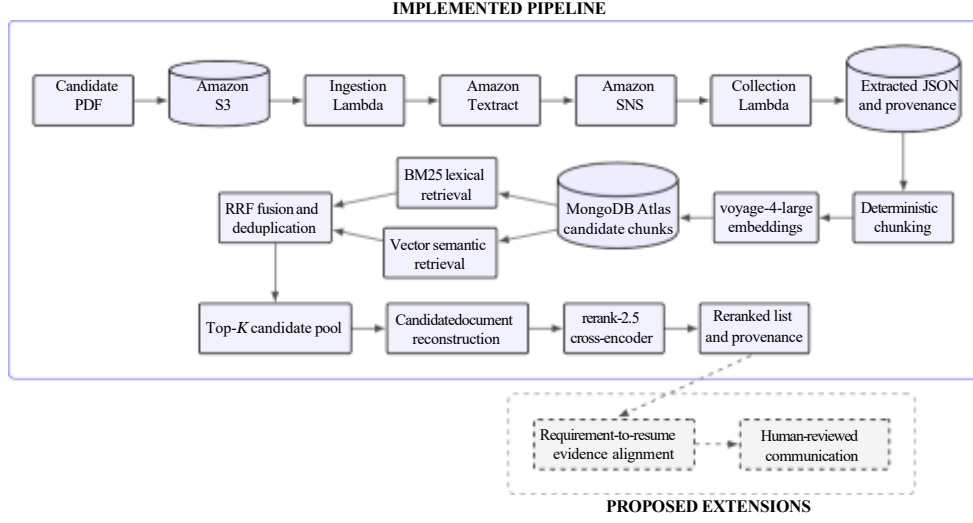


Figure 3.1: Implemented hybrid retrieval architecture with proposed downstream extensions

The implemented path begins with a candidate PDF in Amazon S3. The upload event invokes a Lambda function that starts asynchronous Textract detection. Amazon SNS delivers completion status to a second Lambda function, which retrieves all result pages and writes the extracted JSON to S3. A local processing pipeline converts that JSON into deterministic candidate chunks, generates voyage-4-large embeddings, and stores the chunk records in MongoDB Atlas.

At query time, Atlas Search returns BM25-ranked chunks and Atlas Vector Search returns semantically ranked chunks. RRF combines the two lists, aggregates repeated chunk evidence at candidate level, and produces a unique top- K pool. Candidate resumes are reconstructed from their chunks and evaluated jointly with the job description by rerank-2.5. The dashed extension boundary identifies later evidence alignment and communication functions without representing them as current implementation.

3.5 IMPLEMENTED PROCESS MODEL

The present implementation has two human roles and a sequence of managed services. An authorised operator places a candidate PDF in the configured S3 input prefix and supplies a job description to the retrieval command. The recruiter or project evaluator examines the resulting candidate pool, semantic relevance scores, and rank movement. AWS services perform extraction, while the processing modules perform chunking, indexing, retrieval, fusion, reconstruction, and reranking.

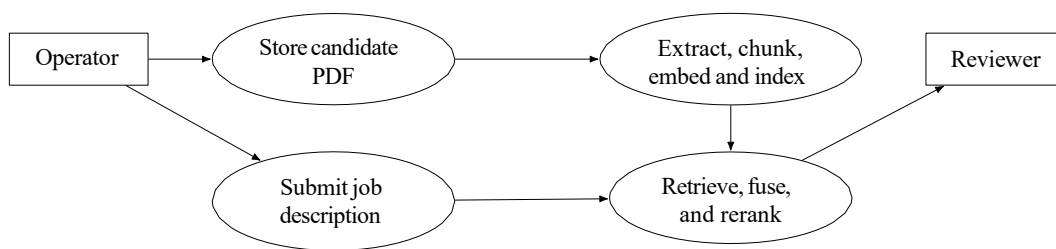


Figure 3.2: Implemented ingestion, retrieval, and review interactions

The model makes responsibility boundaries explicit. Automated components generate a ranked decision-support output; the reviewer retains responsibility for interpreting that output and for any employment decision. Proposed evidence alignment will add source-linked support records to this review step.

Table 3.3: Core data entities

Entity	Principal attributes
Source PDF	S3 bucket, object key, ETag/checksum, upload event metadata
Textextract output	Job ID, pages, blocks, relationships, geometry, status
Candidate chunk	Candidate ID, chunk ID, text, section, page/source reference
Embedding	1024 floating-point components and embedding model identifier
Retrieval result	Branch, chunk rank, candidate ID, raw score, source metadata
Fusion result	Candidate ID, accumulated RRF score, contributing ranks, pool rank
Rerank result	Candidate ID, semantic relevance score, previous rank, new rank
Proposed evidence	Requirement, source span, support state, confidence, provenance

3.6 DATA DESIGN

The implemented design uses two principal stores. Amazon S3 preserves the source PDF and extracted Textextract JSON, while MongoDB Atlas stores one record for each candidate chunk together with embedding and provenance metadata. Retrieval and reranking outputs are written as versionable JSON artefacts. A relational application database is not part of the verified implementation.

Deterministic chunk identifiers and source hashes make repeated processing auditable and support idempotent upserts. Candidate-level reconstruction orders the stored chunks deterministically so the reranker receives a stable document representation. Personally identifying fields are not required to compute RRF or the semantic relevance score.

3.7 MODULE INTERFACES

The repository exposes service boundaries through event payloads, JSON artefacts, configuration, and command-line entry points rather than a public web API. The extraction starter consumes an S3 object-created event and returns a Textract job identifier. The callback handler consumes the SNS completion message, follows every NextToken, and writes the complete response to the extracted prefix.

The indexing command consumes extracted JSON and emits deterministic chunk documents for MongoDB. Retrieval consumes a job description and search limits, invokes both Atlas search paths, and returns ranked chunk records. Fusion converts these records into a unique candidate pool. Reranking consumes that pool, reconstructs candidate text, invokes the configured cross-encoder, and writes semantic relevance scores with preserved retrieval provenance.

Stable candidate identifiers, chunk identifiers, model identifiers, and source references connect the modules. Failures are raised at their originating boundary so that extraction, indexing, retrieval, or reranking can be retried without silently substituting incomplete data.

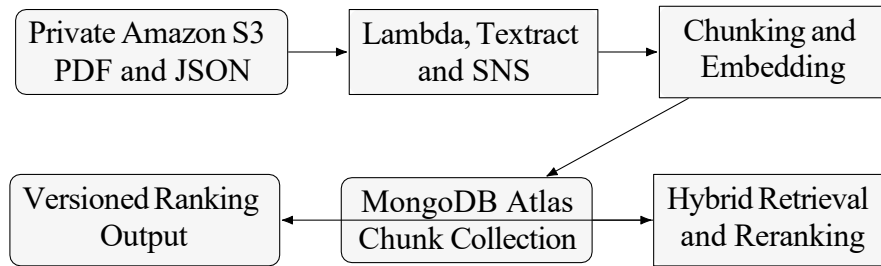


Figure 3.3: Logical deployment view

3.8 DEPLOYMENT AND SECURITY DESIGN

The verified cloud deployment consists of a private S3 bucket, two Lambda functions, Amazon Textract, and an SNS completion topic. Infrastructure configuration connects S3 notifications to the starter Lambda and the Textract service role to SNS. The indexing, retrieval, fusion, and reranking modules use MongoDB Atlas and Voyage AI through protected environment configuration. No Cognito, API Gateway, VPC application tier, or PostgreSQL database is claimed.

Implemented controls visible in configuration include private object storage, service-specific IAM permissions, protected credentials, and provenance-preserving artefacts. Encryption in transit is provided by the managed service endpoints. Retention enforcement, formal deletion workflows, user authentication, and reviewer-facing access control remain deployment controls to be completed before production use.

CHAPTER 4

IMPLEMENTED METHODOLOGY AND PROPOSED EXTENSIONS

4.1 END-TO-END METHOD

The method begins when a PDF resume is placed in Amazon S3. An object-created event starts asynchronous text detection; the completion callback retrieves every Textract response page and preserves the raw JSON. Deterministic chunking converts the extracted blocks into bounded, provenance-bearing units. Each chunk receives a voyage-4-large embedding and is stored in MongoDB Atlas.

For a job description, BM25 lexical retrieval and vector semantic retrieval run independently over the chunk collection. RRF combines their ranked results and aggregates all matching chunks belonging to a candidate before deduplication. The resulting top- K candidate pool is reconstructed into candidate documents and evaluated by rerank-2.5. The implemented output is a reranked list with semantic relevance scores, source provenance, and rank movement. Evidence alignment and communication are extensions of this output rather than completed components.

4.2 RESUME INGESTION AND VALIDATION

The current ingestion path accepts PDF resumes only. Files are stored under the configured S3 input prefix. An S3 ObjectCreated event invokes the starter Lambda, which calls StartDocumentTextDetection and supplies the SNS notification channel. The asynchronous design avoids holding a request open during OCR and allows the Textract job identifier to become the correlation key.

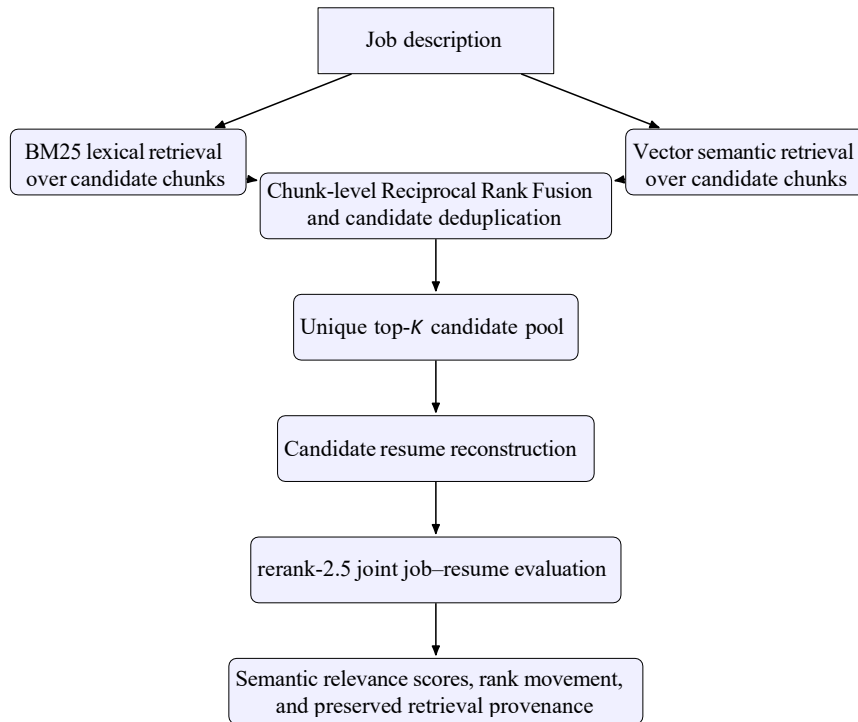


Figure 4.1: Hybrid retrieval and cross-encoder reranking method

Source hashes and deterministic object paths support idempotent downstream processing. The raw PDF is preserved so extracted text can be checked against the original document. DOCX conversion, candidate self-service upload, malware scanning, and profile correction are not part of the verified repository and are therefore excluded from the implemented scope.

Production deployment should retain private bucket policies, encryption, least-privilege roles, and lifecycle rules. These controls complement the application logic without changing the retrieval model.

4.3 DOCUMENT EXTRACTION AND PREPROCESSING

When Textract publishes a successful status to SNS, the callback Lambda calls GetDocumentTextDetection. If the response includes NextToken, the handler continues until all pages are collected. The combined object is written to extracted/<job-id>.json. This retains block type, page number, relationships, geometry, confidence, and text returned by the service.

The repository does not claim that Textract converts a resume into semantic entities such as a normalised education or experience profile. Its verified responsibility is text detection with document provenance. Deterministic preprocessing reads LINE blocks, maintains page associations, recognises configured section headings, and applies bounded fallback windows where a section cannot be identified reliably.

Each chunk records candidate identity, chunk order, page/source information, section label when available, and a stable source hash. These fields support reconstruction and create a foundation for future evidence links.

4.4 JOB-DESCRIPTION QUERY MODELLING

The current system accepts the job description as a text query. The same text is supplied to BM25 lexical retrieval, encoded for vector semantic retrieval, and paired with each reconstructed candidate document during cross-encoder reranking. This consistent query boundary prevents one branch from silently evaluating a different role description.

The query is not automatically converted into mandatory gates or independently weighted requirements in the implemented pipeline. Such structuring is reserved for the proposed evidence-alignment extension. When introduced, requirement records should be confirmed by a recruiter, versioned, and linked to the exact job-description source span.

Query text, model identifiers, top- K limits, branch depths, and RRF parameters should be stored with a screening run. These values are necessary to reproduce the candidate pool and interpret later rank changes.

4.5 EMBEDDING AND VECTOR INDEXING

The processed resume is represented at chunk level. For candidate i , chunk j has an embedding $\mathbf{x}_{ij} \in \mathbb{R}^d$, while the job description has query embedding $\mathbf{q} \in \mathbb{R}^d$. The current configuration uses voyage-4-large with $d = 1024$. Cosine similarity is

$$s_{\text{vec}}(\mathbf{q}, \mathbf{x}_{ij}) = \frac{\mathbf{q}^T \mathbf{x}_{ij}}{\|\mathbf{q}\|_2 \|\mathbf{x}_{ij}\|_2}.$$

MongoDB Atlas stores the text, embedding, candidate ID, chunk ID, section/page provenance, source hash, and model identifier in the `candidate_chunks` collection. Atlas Vector Search performs approximate nearest-neighbour retrieval. A model change requires a compatible index and a separate re-embedding operation; vectors from incompatible spaces must not be mixed.

The design intentionally avoids assuming one vector per resume. Chunk-level records preserve local evidence and allow multiple passages from the same candidate to contribute during fusion.

4.6 HYBRID CANDIDATE RETRIEVAL

MongoDB Atlas Search computes the BM25 branch internally. A standard expression for a query Q and chunk D is

$$s_{\text{BM25}}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \frac{f(t, D)(k_1 + 1)}{f(t, D) + k_1 \left(\frac{1 - b + b|D|/|\overline{D}|}{1 - b} \right)}.$$

The vector branch ranks chunks using s_{vec} . Because the two score scales are not directly comparable, the implementation fuses ranked positions. For candidate c ,

$$s_{\text{RRF}}(c) = \sum_{m \in \{\text{BM25}, \text{vec}\}} \sum_{h \in H_m(c)} \frac{w_m}{60 + r_m(h)},$$

where $H_m(c)$ is every returned chunk of candidate c in branch m . Thus, the implemented `fusion.py` accumulates the contribution of each matching chunk before sorting candidates and deduplicating to candidate IDs. With default equal branch weights, neither retrieval method is assumed superior in advance.

The candidate pool size K is configurable. Increasing K may improve coverage but increases cross-encoder pair evaluations. Retrieval output preserves the source chunk, branch rank, raw branch score, and candidate identifier so the fused result can be inspected.

4.7 CROSS-ENCODER RERANKING

For each retrieved candidate, the implementation loads all stored chunks, orders them deterministically, and reconstructs a candidate document D_c . The configured Voyage cross-encoder jointly evaluates the job description q and this candidate document:

$$s_{\text{rank}}(c \mid q) = R(q, D_c),$$

where R is `rerank-2.5` in the current implementation. The model returns a semantic relevance score used to reorder the top- K pool.

The score is not interpreted as a probability of job success. It is a model-specific relevance signal. Displayed scores are labelled accordingly, and recruiters receive requirement evidence rather than an unsupported percentage claim. Ties and near-ties are presented without artificial precision.

The run output includes the reranker model, input candidate count, output limit, semantic relevance score, retrieval provenance, original rank, and reranking rank. Re-running frozen inputs with the same service version and configuration supports reproducible comparison.

4.8 COMPUTATIONAL AND STORAGE MODELS

Let N be the full candidate corpus and K the hybrid candidate-pool size. If c_x denotes the cost of one query–document cross-encoder evaluation, exhaustive reranking has analytical workload $W_{\text{full}} = Nc_x$, whereas the implemented two-step design has $W_{\text{two}} = Kc_x$. Therefore,

$$\frac{W_{\text{two}}}{W_{\text{full}}} = \frac{K}{N}$$

This is a computational model, not a measured latency or count of HTTP requests. A service may batch several query–document pairs into one API call.

Storage must also be modelled per chunk. If c is the average chunks per candidate, $M = Nc$ is the total number of chunks, d is embedding dimension, b is bytes per component, and m is metadata footprint per chunk, then

$$B_{\text{index}} \approx M(db + m) = Nc(db + m).$$

The expression is an analytical approximation for application data. It does not include MongoDB Atlas index structures, replication, compression, backups, or billing units.

These models clarify the principal trade-off: finer chunking can improve local retrieval evidence but increases index size and the number of ranked chunk results requiring aggregation.

4.9 PROPOSED EVIDENCE ALIGNMENT

The next extension will compare confirmed job requirements with resume spans that retain provenance. It should record the best supporting span, alternatives, support state, confidence, and source location. “Not evidenced” must remain a valid result; the service should not infer experience from an unrelated term. This extension will consume the reranked list while leaving its semantic relevance scores unchanged.

Table 4.1: Proposed explanation-record fields

Field	Purpose
Candidate and job IDs	Identify the evaluated pair
Retrieval and rerank ranks	Preserve both ordering stages
Requirement ID and text	Identify the criterion being assessed
Evidence span and source	Link the assessment to the resume source
Support classification	Supported, partial, not evidenced, or review required
Model and lexicon versions	Enable reproducibility
Timestamp and audit reference	Support governance and investigation

A language model, if used in this proposed layer, should receive only the structured record and be constrained to verbalise its contents. Automated validation should reject output containing unsupported entities or claims.

4.10 PROPOSED ALTERNATIVE-OPENING MATCHING

Candidates may be better suited to an opening other than the one for which they initially applied. A proposed extension can evaluate a consented candidate against multiple active roles using the same retrieval, reranking, and evidence rules. Alternative-opening suggestions should be shown to an authorised recruiter only when consent and organisational policy allow reuse of the profile.

The recommendation would not use a generic collaborative-filtering profile. It would be derived from candidate evidence relative to the requirements of each opening. A recruiter would see both relevance and requirement evidence, and candidate communication would occur only after human approval.

This planned capability can improve talent reuse while preserving governance. It also avoids presenting a candidate with a vague suggestion unsupported by an actual role and evidence comparison.

4.11 PROPOSED CANDIDATE COMMUNICATION

After evidence alignment is validated, skill gaps may be derived from the requirement-level evidence record. A gap may mean no evidence was found, available evidence was partial, or the experience context did not meet the stated requirement. The future system should distinguish these states so candidates are not told they lack a skill merely because the resume did not describe it clearly.

Approved learning resources can be mapped to canonical skills and requirement categories. Recommendations are advisory and do not guarantee future selection. Candidate-facing language avoids disclosing internal ranking details, other applicants' data, or sensitive recruiter notes. Feedback is constructive, role specific, and constrained by the source record.

Future recruiter and candidate views should share a common evidence foundation while exposing different authorised information. This prevents inconsistencies between an internal explanation and an external message.

4.12 OUTPUT, AUDITABILITY, AND OBSERVABILITY

The implemented output preserves the candidate identifier, RRF position, contributing retrieval evidence, reranker model, semantic relevance score, and rank movement. These fields support comparison of candidate generation and reranking without interpreting the score as a hiring outcome. JSON output also provides a stable input for later reviewer interfaces.

Operational observability should cover upload failures, Textract status, callback pagination, embedding failures, Atlas query errors, fusion parameters, and reranker failures. Logs should avoid raw resume content unless necessary for controlled diagnosis. Source hashes and model identifiers allow an output to be connected to its inputs.

Future audit functions can add actor, action, access, correction, export, deletion, and communication events. Such governance records are distinct from the ranking algorithm and should be implemented before a production recruitment workflow is enabled.

CHAPTER 5

IMPLEMENTATION, PILOT RESULTS, AND VALIDATION PLAN

5.1 IMPLEMENTED COMPONENTS

The repository contains a working ingestion and extraction path using S3, Lambda, Textract, and SNS. It also contains deterministic chunking, Voyage embedding, MongoDB Atlas indexing, BM25 retrieval, vector retrieval, RRF fusion, candidate reconstruction, and cross-encoder reranking. Configuration and structured outputs retain model and source metadata across these boundaries.

The extraction handler preserves the asynchronous Textract response rather than presenting an inferred semantic profile. The retrieval implementation operates over candidate chunks, and fusion.py accumulates every returned chunk contribution at candidate level before deduplication. The reranking implementation uses rerank-2.5, preserves the earlier retrieval record, and reports rank movement.

Evidence alignment, explanation generation, alternative-opening matching, and candidate communication are design extensions. They are included to show how the current provenance can support a fuller human-review workflow, but they are not counted as delivered implementation.

5.2 TEST STRATEGY

Testing is organised by module and interface. Unit tests cover event parsing, Textract pagination, deterministic chunk boundaries, identifiers, reconstruction, fusion arithmetic, and result schemas. Integration tests cover S3 event handling, SNS callback processing, MongoDB writes, Atlas Search, Atlas Vector Search, Voyage embeddings, and Voyage reranking.

The repository test collection contains 68 cached cases and reports no cached failures in the inspected state. This indicates useful regression coverage for code paths represented by the suite; it is not a substitute for relevance labels, production load testing, or an independent security assessment.

End-to-end tests should begin with a PDF and job description and finish with an ordered output record. Negative cases should include malformed events, failed Textract jobs, paginated callbacks, duplicate processing, missing chunks, unavailable external services, incompatible embedding dimensions, and reranker timeouts.

5.3 EVALUATION METRICS

Extraction quality will be measured using precision, recall, and F1 for detected lines or governed target fields, with separate review of reading order and page provenance. Retrieval will be evaluated using Recall@K, BM25/vector overlap, hybrid coverage, and nDCG@K or MRR when relevance labels exist. Reranking will be evaluated using nDCG@K, Kendall's τ , Spearman correlation where appropriate, top-K agreement, and rank movement. Latency will be reported separately for extraction, embedding, lexical search, vector search, fusion, reconstruction, and reranking.

After the evidence extension is implemented, evidence coverage, faithfulness, and unsupported-claim rate will be measured. Reproducibility will be checked by repeated runs over frozen inputs. System evaluation will record storage growth, API usage, cost, and stage-level latency. Fairness-related analysis requires a governed dataset and review protocol; protected attributes must not be used as ranking inputs.

Table 5.1: Planned evaluation matrix

Dimension	Measure	Purpose
Extraction	Precision, recall, F1, reading order	Verify detected text and provenance
Retrieval	Recall@ K , overlap, coverage, nDCG/MRR	Verify hybrid candidate generation
Reranking	nDCG@ K , τ , Spearman, agreement	Compare ordering with labels
Explanation	Evidence coverage, faithfulness, unsupported rate	Validate proposed extension
System	Stage latency, storage, API usage, cost	Assess operational behaviour
Stability	Repeated and counterfactual runs	Detect fragile behaviour

5.4 OBSERVED PILOT RETRIEVAL

A recorded pilot query returned eight unique candidates from BM25 and eight from vector search. Five candidate identifiers occurred in both lists. The union contained eleven unique candidates, demonstrating how the hybrid pool retained candidates contributed by either branch for that query.

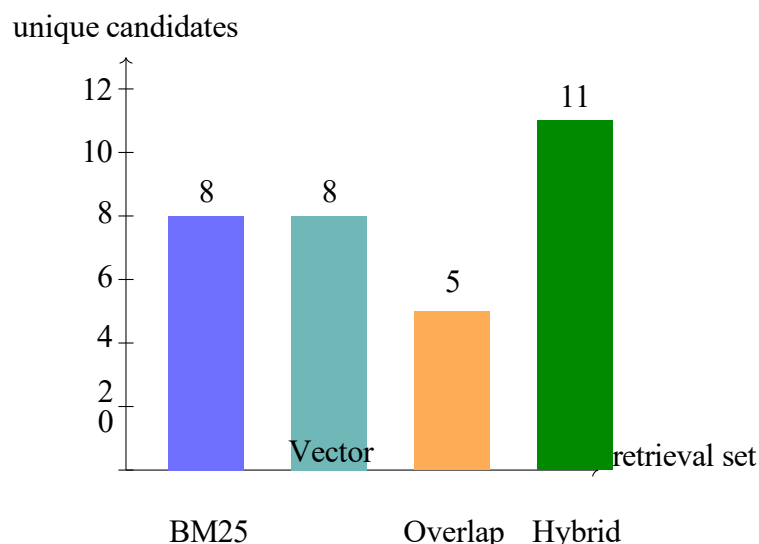


Figure 5.1: Unique-candidate coverage observed for one pilot job description

The observation is a pipeline check, not a corpus-level quality claim. Without relevance judgements it does not establish $\text{Recall}@K$ or ranking accuracy. Its value is that the identifiers and overlap are measured directly from current retrieval output.

5.5 OBSERVED RERANK MOVEMENT

A stored reranking output accepted eleven candidates and returned five. For five anonymised candidates, the retrieval-to-reranking movements were $1 \rightarrow 1$, $4 \rightarrow 2$, $10 \rightarrow 3$, $2 \rightarrow 4$, and $9 \rightarrow 5$. The slope chart records how joint job–document evaluation changed order within the retrieved pool.

Rank movement is expected because the cross-encoder jointly evaluates query and reconstructed candidate text. It should be interpreted as a behaviour of the configured model, not by itself as proof of candidate quality. Labelled judgements are required for effectiveness metrics.

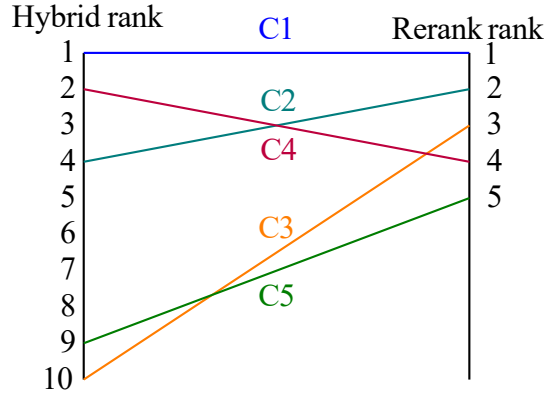


Figure 5.2: Retrieval-rank to reranking-rank movement in the recorded pilot output

5.6 EXPERIMENTAL PROTOCOL

The full evaluation dataset should contain consented or appropriately de-identified PDF resumes and representative job descriptions. Reviewers should annotate candidate relevance and, once evidence alignment exists, requirement-support spans using a shared rubric. Disagreements should be measured and adjudicated rather than silently collapsed.

Baselines should include BM25 alone and vector search alone. The hybrid pool and cross-encoder output should be compared under the same corpus, queries, and retrieval depths. Ablations should examine chunking strategy, branch depth, RRF weights, top- K , candidate reconstruction, and reranking. Model and configuration versions must be fixed before final measurement.

Results should include confidence intervals or repeated-query variation where appropriate. Failure analysis should distinguish extraction errors, lexical mismatch, semantic mismatch, fusion effects, reconstruction truncation, and ambiguous job descriptions.

5.7 RISK, ETHICS, AND MITIGATION

The most serious risk is over-reliance on an automated ranking. Mitigation includes human review, explicit semantic-relevance labelling, preserved provenance, and no autonomous rejection. Bias may enter through language, model pretraining, institution proxies, locations, or the job description. Future evidence alignment should expose supporting passages without implying that a passage is a complete employment assessment.

Privacy risks arise from centralised resume storage and external model calls. Controls include encryption, strict service roles, data minimisation, retention, deletion, and an explicit review of information sent to external services. Future generated feedback must be evidence constrained and validated before communication.

Technical risks include OCR failure, model drift, vector-index inconsistency, duplicate events, service availability, and cost. Versioned artefacts, idempotent processing, monitoring, compatible index definitions, and cost metrics support detection and recovery.

5.8 ACHIEVEMENTS AND SUCCESS CRITERIA

The project has achieved a working path from PDF ingestion through asynchronous extraction, deterministic chunking, chunk-level embeddings, hybrid retrieval, RRF fusion, candidate reconstruction, and cross-encoder reranking. The pilot outputs demonstrate complementary candidate coverage and observable rank movement while preserving the inputs needed for inspection.

The next evaluation milestone is quantitative validation on labelled data. Success criteria include reliable extraction with provenance, strong Recall@K at an operationally reasonable pool size, useful nDCG or MRR relative to branch baselines, stable repeated runs, complete model and parameter metadata, and acceptable stage-specific latency and cost.

The proposed evidence layer will add further criteria: evidence coverage, faithfulness, unsupported-claim rate, reviewer usability, and safe human-approved communication. These criteria extend the current system without overstating the meaning of semantic relevance scores.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project implements a coherent pipeline for AI-assisted resume screening. Event-driven S3, Lambda, Textract, and SNS processing converts a PDF into provenance-bearing JSON. Deterministic chunking and voyage-4-large embeddings create a searchable MongoDB Atlas collection. BM25 lexical retrieval and vector semantic retrieval contribute complementary evidence, while RRF combines ranked chunk results into a unique candidate pool.

The selected candidates are reconstructed and evaluated jointly with the job description by rerank-2.5. Semantic relevance scores, retrieval provenance, and rank movement show how candidate generation differs from final reranking. In the pilot, both retrieval branches returned eight unique candidates, five overlapped, and the hybrid union retained eleven. These results establish a practical foundation for labelled evaluation and evidence-grounded review.

6.2 FUTURE WORK

Future work will create a governed labelled dataset and compare BM25-only, vector-only, hybrid, and reranked outputs. Branch depth, RRF weights, chunk size, pool size, and reconstruction limits will be calibrated using Recall@K, nDCG, MRR, rank correlation, latency, storage, API use, and cost.

Requirement-to-resume evidence alignment is the next functional extension. It will connect confirmed requirements to preserved source spans and support reviewer inspection. Alternative-opening matching and candidate communication will follow after evidence and governance controls are validated. Later work may add multilingual resumes, improved table reading, organisation-specific terminology, privacy-preserving deployment, and applicant-tracking integration.

REFERENCES

1. Amazon Web Services (2026), “Calling Amazon Textract asynchronous operations,” *AWS Textract Developer Guide*, accessed October 2026.
2. Amazon Web Services (2026), “What is Amazon Textract?,” *AWS Textract Developer Guide*, accessed October 2026.
3. Bhaia, V., Rawat, P., Kumar, A. and Shah, R. R. (2019), “End-to-end resume parsing and finding candidates for a job description using BERT,” arXiv:1910.03089.
4. Zhu, C., Zhu, H., Xiong, H., Ma, C., Xie, F., Ding, P. and Li, P. (2018), “Person-job fit: Adapting the right talent for the right job with joint representation learning,” *ACM Transactions on Management Information Systems*, Vol. 9, No. 3, pp. 1–17.
5. Dastin, J. (2018), “Amazon scraps secret AI recruiting tool that showed bias against women,” *Reuters*, 10 October 2018.
6. Decorte, J.-J., Van Haute, J., Demeester, T. and Develder, C. (2021), “JobBERT: Understanding job titles through skills,” *Proceedings of FEAST Workshop at ECML-PKDD*.
7. Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019), “BERT: Pre-training of deep bidirectional transformers for language understanding,” *Proceedings of NAACL-HLT*, pp. 4171–4186.
8. Government of India (2023), *The Digital Personal Data Protection Act, 2023*, Ministry of Electronics and Information Technology.
9. Karpukhin, V., O’guz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D. and Yih, W.-T. (2020), “Dense passage retrieval for open-domain question answering,” *Proceedings of EMNLP*, pp. 6769–6781.
10. Kesim, E. and Deliahmetoglu, A. (2023), “Named entity recognition in resumes,” arXiv:2306.13062.
11. Khelkhal, K. and Lanasri, D. (2025), “Smart-Hiring: An explainable end-to-end pipeline for CV information extraction and job matching,” arXiv:2511.02537.
12. Lin, J. and Ma, X. (2022), “A few brief notes on DeepImpact, COIL, and a conceptual framework for information retrieval techniques,” arXiv:2106.14807.
13. Cormack, G. V., Clarke, C. L. A. and Buettcher, S. (2009), “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” *Proceedings of ACM SIGIR*, pp. 758–759.
14. Robertson, S. and Zaragoza, H. (2009), “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, Vol. 3, No. 4, pp. 333–389.
15. MongoDB Inc. (2026), “MongoDB Search and Vector Search documentation,” accessed October 2026.

16. Lundberg, S. M. and Lee, S.-I. (2017), “A unified approach to interpreting model predictions,” *Advances in Neural Information Processing Systems*, pp. 4765–4774.
17. Mujtaba, D. F. and Mahapatra, N. R. (2024), “Fairness in AI-driven recruitment: Challenges, metrics, methods, and future directions,” arXiv:2405.19699.
18. New York City Council (2023), *Local Law 144 of 2021: Automated Employment Decision Tools*, New York City Administrative Code.
19. Nogueira, R. and Cho, K. (2019), “Passage re-ranking with BERT,” arXiv:1901.04085.
20. Qin, C., Zhu, H., Xu, T., Zhu, C., Jiang, L., Chen, E. and Xiong, H. (2018), “Enhancing person-job fit for talent recruitment: An ability-aware neural network approach,” *Proceedings of ACM SIGIR*, pp. 25–34.
21. Reimers, N. and Gurevych, I. (2019), “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” *Proceedings of EMNLP-IJCNLP*, pp. 3982–3992.
22. Ribeiro, M. T., Singh, S. and Guestrin, C. (2016), “Why should I trust you?: Explaining the predictions of any classifier,” *Proceedings of ACM SIGKDD*, pp. 1135–1144.
23. Voyage AI (2026), “Embedding and reranking models,” documentation, accessed October 2026.
24. Vrang, M. le, Papantoniou, A., Pauwels, E., Fannes, P., Vandenstein, D. and De Smedt, J. (2014), “ESCO: Boosting job matching in Europe with semantic interoperability,” *Computer*, Vol. 47, No. 10, pp. 57–64.
25. Webster, K. T. (2025), “Fairness is not enough: Auditing competence and intersectional bias in AI-powered resume screening,” arXiv:2507.11548.
26. Weichselbraun, A., Brasoveanu, R., Kuntschik, P. and Nixon, L. (2022), “Building knowledge graphs and recommender systems for suggesting reskilling and upskilling options from the web,” *Information*, Vol. 13, No. 11, Article 510.
27. Zhang, M., Jensen, K. N., Sonniks, S. D. and Plank, B. (2022), “SkillSpan: Hard and soft skill extraction from English job postings,” *Proceedings of NAACL-HLT*, pp. 4962–4984.
28. Zhang, M., van der Goot, R., Kan, M.-Y. and Plank, B. (2024), “NNOSE: Nearest neighbor occupational skill extraction,” *Proceedings of EACL*

A Hybrid Retrieval and Cross-Encoder Reranking Architecture for AI-Assisted Resume Screening and Talent Acquisition

Daniel Leve Manickam D A

230701060@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Darshan M

230701061@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Dayanithi V

230701064@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Deepa S

230701065@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Dr. J. Jinu Sophia

jinasophia.j@rajalakshmi.edu.in

Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Abstract—Large applicant collections require candidate generation to be efficient without reducing screening to exact keyword overlap. This paper describes an implemented retrieval-and-reranking pipeline for PDF resumes. An Amazon S3 event invokes asynchronous Amazon Textract processing, and the resulting text and layout blocks are retained as provenance-bearing JSON. A deterministic section-aware procedure divides each resume into candidate chunks. Voyage voyage-4-large produces 1024-dimensional representations, while MongoDB Atlas independently performs BM25 lexical retrieval and vector semantic retrieval. Reciprocal Rank Fusion combines the two chunk rankings and deduplicates them into a candidate pool. Candidate documents are reconstructed from stored chunks and jointly evaluated with the job description by the configured Voyage rerank-2.5 cross-encoder. Its output is treated as a semantic relevance score rather than a hiring probability. A repository-backed pilot illustrates complementary retrieval coverage and rank movement, but does not establish ranking accuracy. Requirement-to-resume evidence alignment, explainable recommendations, alternative-opening matching, and candidate communication are presented only as proposed extensions for human-reviewed decision support.

Keywords—hybrid retrieval, BM25 lexical retrieval, vector semantic retrieval, Reciprocal Rank Fusion, candidate chunk, cross-encoder reranking.

I. INTRODUCTION

Recruitment screening is an information-retrieval problem before it is a decision problem. A technical vacancy may attract resumes that describe similar competence with different vocabulary, structure, and detail. Exact-term filtering can miss relevant applicants, while applying an expensive joint language model to every job–resume pair does not scale gracefully. The practical requirement is therefore to retrieve broadly, rerank selectively, and retain enough provenance for human review.

Bi-encoders make corpus search efficient because documents can be embedded before a query arrives [1], [2]. Their independent representations, however, do not model every interaction between a job description and a complete resume. Cross-encoders provide richer pairwise comparison but require

evaluation of each selected pair [3]. The implemented system joins these approaches with lexical retrieval: MongoDB Atlas Search supplies BM25 rankings, Atlas Vector Search supplies semantic rankings, and Reciprocal Rank Fusion (RRF) merges them before cross-encoder reranking [4].

This work contributes: 1) an event-driven, PDF-only extraction path that preserves Textract page and block data; 2) deterministic, section-aware candidate chunks with idempotent indexing; 3) chunk-level BM25 and vector retrieval with implementation-aligned candidate aggregation; 4) reconstruction and rerank-2.5 scoring that preserves retrieval provenance; and 5) an evaluation framework that separates observed pilot behaviour from unmeasured quality. Evidence alignment and communication are future extensions rather than implemented claims.

II. BACKGROUND AND RELATED WORK

A. Resume Document Intelligence

Resume processing begins with document recovery rather than semantic judgement. Layout, reading order, repeated headings, and page boundaries affect later retrieval. Amazon Textract supports asynchronous text detection for documents stored in S3 and reports completion through Amazon SNS [5]. The current system preserves the returned text, blocks, page numbers, and geometry; it does not claim that Textract directly produces verified skills, education, or experience entities. Research on resume parsing and skill-span extraction motivates later structured analysis, but those capabilities must be evaluated separately [6], [7].

B. Lexical and Dense Retrieval

BM25 rewards query terms according to within-document frequency, document length, and corpus rarity [8]. It is useful when a job description contains exact technologies, certifications, or role names. Dense retrieval maps differently worded but related passages into a shared vector space [1], [2]. Neither signal is universally sufficient: lexical search can miss paraphrases, whereas dense search can underweight exact constraints.

C. Hybrid Retrieval and RRF

Hybrid retrieval retains both signals without forcing raw scores onto a common scale. RRF combines ordinal positions

[4]. Here, the ranked objects are resume chunks: all retrieved chunks contribute to their candidate, and the fused result is then deduplicated. This differs from fusing one pre-aggregated score per resume.

D. Cross-Encoder Reranking

BERT reranking demonstrates the value of jointly encoding a query and a candidate passage [3]. The deployed reranker similarly receives a job description and an array of reconstructed candidate documents. Its score orders semantic relevance within the retrieved pool; it is not calibrated as a probability of hiring success and is not combined numerically with the RRF score.

E. Explainability, Fairness and Human Oversight

Post-hoc explanation methods can indicate influential features [9], [10], but employment screening needs verifiable source evidence, not merely fluent rationales. Removing explicit identity fields also does not remove institutional, geographic, or career-history proxies [11]. Accordingly, proposed explanations will be constrained by stored resume evidence, and recruiters will remain responsible for requirements, corrections, and final decisions.

III. PROBLEM FORMULATION AND MATHEMATICAL MODEL

A. Candidate Chunk Representation

Let candidate c_i contain an ordered set of chunks $D_i = \{x_{i1}, \dots, x_{in_i}\}$. The implementation embeds each chunk and the job description in $\mathbb{R}^d$, with $d = 1024$ for the configured voyage-4-large model [12], [25]. Document chunks use the model's document input mode and the job description uses query mode.

B. Vector Similarity

For job-description embedding q and chunk embedding x_{ij} , vector retrieval is represented by cosine similarity:

$$s_{\text{vec}}(q, x_{ij}) = \frac{q^T x_{ij}}{\|q\|_2 \|x_{ij}\|_2}. \quad (1)$$

MongoDB Atlas Vector Search executes the approximate-nearest-neighbour query over stored chunk vectors [13].

C. BM25 Retrieval

For query terms $t \in q$ and candidate chunk x , the standard BM25 form is

$$s_{\text{BM25}}(q, x) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, x)(k_1 + 1)}{f(t, x) + k_1(1 - b + b|x|/\text{avgdl})}. \quad (2)$$

MongoDB Atlas Search computes the deployed score internally; the application consumes only the returned chunk order and does not manually normalise BM25 values [14].

D. Reciprocal Rank Fusion

Let L_m be the ranked chunks from method $m \in \{\text{lex}, \text{vec}\}$. The exact candidate-level behavior in fusion.py is

$$S_{\text{RRF}}(c_i) = \sum_m \sum_{\substack{x \in L_m \\ \text{cand}(x)=c_i}} \frac{1}{60 + r_m(x)}. \quad (3)$$

Thus, every retrieved chunk contributes; method weights are one. The best per-method rank retained for provenance is

$$r_m^*(c_i) = \min_{x \in L_m: \text{cand}(x)=c_i} r_m(x), \quad (4)$$

and remains null when that method did not retrieve the candidate. Sorting S_{RRF} and truncating after candidate deduplication yields the pool $\mathcal{C}_K$.

E. Cross-Encoder Reranking

All MongoDB chunks for c_i are ordered by chunk_index and reconstructed as D_i^* . The configured model returns

$$s_{\text{rank}}(c_i | q) = R(q, D_i^*), \quad c_i \in \mathcal{C}_K, \quad (5)$$

where R is rerank-2.5 [15], [26]. The final order is descending in this semantic relevance score. Rank movement is recorded as $\Delta r_i = r_i^{(2)} - r_i^{(3)}$; positive values indicate movement toward rank one.

F. Computational Workload

If N is the corpus size, K the retrieved pool, and c_x the conceptual cost of one query–document evaluation,

$$W_{\text{full}} = N c_x, \quad W_{\text{two}} = K c_x, \quad \frac{W_{\text{two}}}{W_{\text{full}}} = \frac{K}{N}. \quad (6)$$

This is an analytical pair-workload model, not measured latency or API-call count. The implementation submits the candidate–document array in one rerank request.

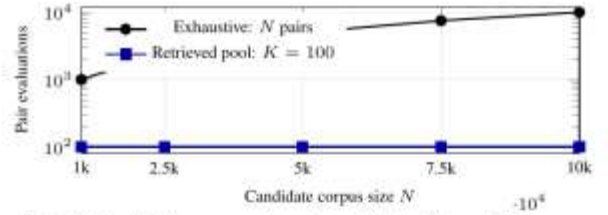


Figure 1. Analytical cross-encoder pair workload as the candidate corpus grows, with a fixed retrieved pool of $K = 100$.

G. Storage Model

For N candidates, average c chunks per candidate, $M = Nc$ total chunks, dimension d , b bytes per component, and metadata footprint m per chunk,

$$B_{\text{index}} \approx M(db + m) = Nc(db + m). \quad (7)$$

This analytical approximation excludes MongoDB index structures, replication, source JSON, and billing overhead.

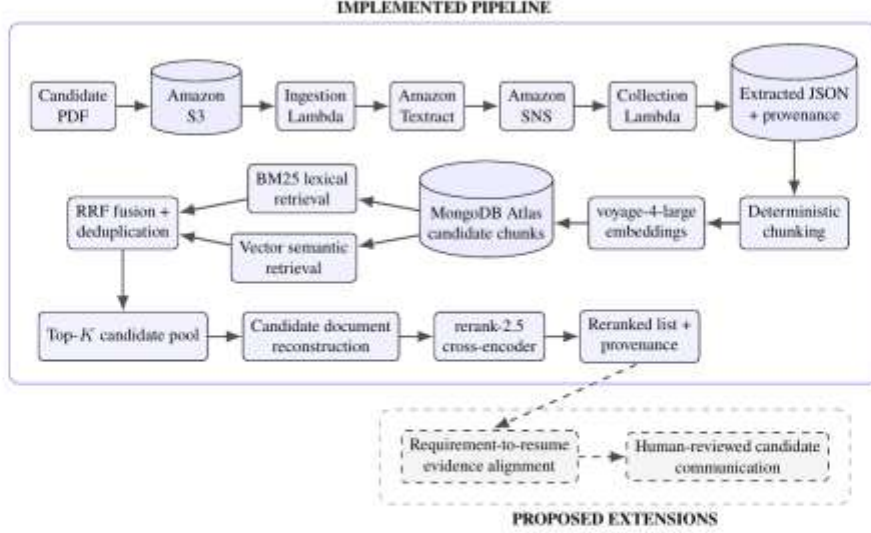


Figure 2. Hybrid retrieval and cross-encoder reranking pipeline with proposed evidence and communication extensions.

IV. SYSTEM ARCHITECTURE

A. Overall Architecture

Fig. 2 separates executable components from future extensions. The implemented path ends with a provenance-bearing reranked list. It contains no orchestrator, autonomous agents, Cognito, API Gateway, VPC, relational database, or DOCX path. The architecture is a retrieval decision-support pipeline rather than an autonomous hiring system.

B. PDF Ingestion and Textract Extraction

The S3 bucket accepts events only for PDF keys under incoming/. An object-created event invokes a Lambda function that calls StartDocumentTextDetection and supplies an SNS completion channel. A second Lambda consumes the completion message, follows every NextToken returned by GetDocumentTextDetection, and writes extracted/<job-id>.json. The record contains concatenated line text, the complete Textract block array, page count, original S3 location, processing timestamp, and pipeline version. Failed jobs are written separately.

C. Deterministic Chunking and Indexing

The chunker recognizes canonical headings for summary, skills, experience, projects, education, certifications, achievements, publications, languages, volunteering, and interests. It retains a header chunk and splits oversized sections at line boundaries with a configured 1200-character limit. If no headings are detected, it uses 1000-character windows with 100-character overlap. Chunk identifiers, section labels, ordinal positions, source keys, and page ranges are deterministic. A SHA-256 text hash makes unchanged re-indexing a no-op; changed content replaces stale candidate chunks.

Each chunk is embedded using input_type=document; the job description uses input_type=query. MongoDB stores the text, vector, candidate identifier, content hash, section, page range, and embedding configuration. Operational indexes

support candidate lookup and idempotency, while Atlas Search and Vector Search indexes support retrieval.

D. Hybrid Candidate Retrieval

The lexical pipeline applies the Atlas \$search text operator to chunk text and retains the returned BM25 order. The vector pipeline applies \$vectorSearch to the 1024-dimensional embedding field, using ten times the requested limit as the approximate-search candidate count, capped at 10,000. Both branches currently request 15 chunk hits by default. RRF then accumulates rank contributions, preserves matched chunk identifiers and section labels, deduplicates by candidate identifier, and returns up to 15 candidates.

E. Candidate Reconstruction and Reranking

For each retrieved candidate, all MongoDB chunks are fetched and ordered by chunk_index. Section labels are inserted into a reconstructed profile, which is conservatively limited to 30,000 characters. The implementation passes the job description and valid candidate documents to one Voyage rerank request. rerank-2.5 is the selected model for the current implementation; it is not described as the latest model. The output records rerank rank, semantic relevance score, rank change, reconstructed chunk identifiers, original source key, and all retrieval provenance. RRF and reranker scores remain separate because their scales and meanings differ.

F. Proposed Evidence and Communication Extensions

A future evidence layer will align each confirmed job requirement with supporting resume spans and their page/block provenance. It will distinguish supported, partial, absent, and review-required states, and prevent an explanation generator from introducing claims without stored evidence. Alternative-opening matching and candidate communication will consume this reviewed record. They are design directions, not current repository capabilities.

Table 1. Implemented component contracts and retained outputs

Operation	Main components	Retained output
Extraction	S3, Lambda, Textract, SNS	Text, blocks, pages, geometry, source key
Indexing	Chunker, Voyage, MongoDB	Chunk ID, section, page range, hash, embedding
Retrieval	BM25, vector search, RRF	Method ranks, fused score, matched chunks
Reranking	Reconstruction, rerank-2.5	Relevance score, rank change, provenance

V. IMPLEMENTATION

A. AWS S3 and Textract

AWS CloudFormation declares the infrastructure. The retained S3 bucket blocks public access and applies AES-256 encryption. Separate IAM roles restrict PDF reads, Textract initiation, SNS publication, result collection, and extracted or failed JSON writes.

B. Voyage and MongoDB Integration

The embedding client batches texts, validates query/document modes, retries transient failures, and avoids logging resume text. The indexer derives a stable candidate identifier from the PDF filename and stores chunk-level embeddings. Table 1 summarises the data retained across the pipeline.

C. Reranking Output Contract

The reranking worker reads a job description and its retrieval JSON, reports invalid reconstructions, sorts valid results by relevance score, and writes the reranked output. Its companion summary records input/output counts, failures, processing time, and score distribution.

Repository tests cover deterministic chunking, page propagation, content hashes, multi-chunk RRF, missing-rank preservation, top- K truncation, and reconstruction. The uploaded cache lists 68 collected cases with no cached failures.

VI. EVALUATION METHODOLOGY

A. Dataset and Experimental Setup

The available artefacts contain one backend-engineer job description and an executed retrieval/reranking output. This pilot is suitable for checking data flow, provenance, and rank transformations, but not for estimating general performance. A formal dataset should include consented, de-identified PDF resumes, multiple job families, and independent recruiter relevance labels. Train, tuning, and test partitions must prevent the same candidate or near-duplicate resume from crossing partitions.

B. Extraction and Retrieval Evaluation

Extraction precision, recall, and F1 will be calculated against page-level text and structure annotations. For a recruiter-labelled relevant set $\mathcal{R}_q$, retrieval coverage is

$$\text{Recall}@K(q) = \frac{|\mathcal{C}_K(q) \cap \mathcal{R}_q|}{|\mathcal{R}_q|}, \quad (8)$$

BM25/vector overlap and hybrid union measure complementarity, while MRR and nDCG@ K require graded or ordered relevance labels. The present pilot reports only set counts because no golden labels were found.

C. Reranking and Explanation Evaluation

With recruiter orderings, reranking will be evaluated using nDCG@ K , Kendall’s τ , Spearman correlation, and top- K agreement. Rank movement is descriptive and cannot by itself prove candidate quality. Proposed explanations will be tested for requirement coverage, evidence faithfulness, and unsupported-claim rate. A displayed claim is faithful only when its cited resume span and provenance directly support it.

D. Latency, Storage, API Usage and Cost

Future controlled runs will report extraction time, embedding time, lexical and vector retrieval time, fusion time, reconstruction time, reranking time, API usage, storage growth, and cost. Values will be distributions over repeated workloads rather than single convenient observations. The stored pilot execution time may verify completion, but is not used as a latency benchmark because network conditions, caching, corpus size, and hardware were not controlled.

VII. PRIVACY, FAIRNESS, SECURITY AND LIMITATIONS

Implemented controls include an S3 public-access block, server-side encryption, separate least-privilege AWS roles, environment-based service credentials, source and chunk provenance, deterministic identifiers, and avoidance of full resume text in logs. These are technical controls, not a complete governance programme. Authentication, end-user authorisation, retention enforcement, consent management, deletion workflows, and formal audit review remain to be integrated and tested.

The infrastructure template applies S3-managed encryption and restricts its Lambda roles to the required operations. AWS guidance distinguishes these storage protections from client-side data handling and key-governance decisions [23]. Similarly, Atlas Search and Vector Search protect retrieval functionality only when database credentials, network access, and collection privileges are configured correctly [13], [24]. Production deployment will require secret rotation, environment separation, and auditable access reviews beyond the code examined here.

Resumes contain direct identifiers and proxy attributes. Future evaluation should examine whether institution names, location, employment gaps, or writing style cause rank changes unsupported by job-relevant evidence [11]. Counterfactual checks must be paired with competence-preserving controls; invariance alone is not fairness. Recruiters must be able to correct extraction errors, inspect evidence, and override system order. No candidate should be rejected solely from a semantic relevance score.

The pilot has several limitations: one job description, a small candidate pool, no relevance labels, no extraction ground truth, and no controlled performance benchmark. BM25 settings are managed by Atlas Search, vector recall depends on index configuration, and section detection relies on recognisable headings. RRF can reward candidates with several retrieved chunks, and truncated reconstructed documents may omit late content. These limitations require further study.

VIII. RESULTS AND DISCUSSION

The retrieval output used 15 lexical chunk hits and 15 vector chunk hits. These represented eight unique candidates in each branch, with five shared candidates and an eleven-candidate union (Fig. 3). The result demonstrates complementarity for this query: three candidates were lexical-only and three were vector-only. It does not show that hybrid retrieval improves Recall@ K , since relevance labels are unavailable.

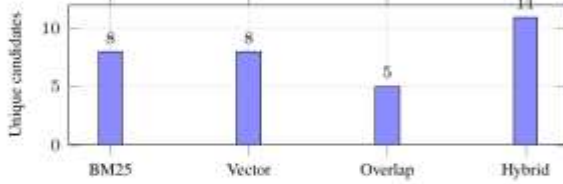


Figure 3. Unique-candidate coverage observed for the backend-engineer pilot query.

The stored reranking artefact received eleven candidates and returned five without reconstruction failures. Fig. 4 shows changes from hybrid ranks 1, 4, 10, 2, 9 to reranked positions 1, 2, 3, 4, 5. One vector-only candidate moved from tenth to third, while the initial leader remained first. This verifies reordering, not correctness; recruiter labels are required.

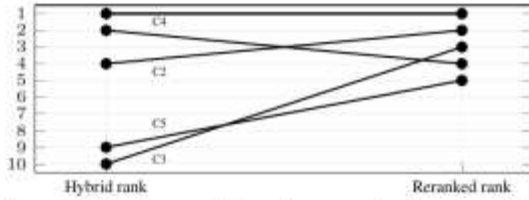


Figure 4. Retrieval-to-reranking rank movement for five anonymised candidates in the pilot output.

A. Planned Comparative Experiments

The first controlled experiment will compare BM25-only, vector-only, and fused retrieval under identical chunking, corpus, and retrieval depths. Recall@ K will be the primary candidate-generation measure, supplemented by MRR and nDCG where graded labels exist [19]. An ablation will then replace candidate-level multi-chunk accumulation with a best-chunk-only rule. This isolates whether repeated evidence across sections improves retrieval or merely rewards longer resumes.

The reranking experiment will hold the retrieved pool fixed and compare its order before and after cross-encoder evaluation. At least two independent recruiters will label a common subset using a written rubric. Inter-rater agreement will be reported before the system is compared with either judgement. Kendall's τ and Spearman's ρ will describe order association [20], [21], while top- K agreement will expose changes within the operational shortlist. Score thresholds will not be selected from the test set.

Extraction will be evaluated separately because downstream ranking cannot recover text that was never detected. Page-level annotations will distinguish missing lines, incorrect reading order, and wrong page provenance. Duplicate uploads will test

content-hash idempotency, and modified versions will verify stale-chunk replacement. These tests focus on system behavior rather than inferring document quality from successful API responses.

System experiments will repeat each workload after a warm-up period and report medians together with dispersion. S3 event delivery, Textract completion, embedding, retrieval, and reranking will be timed independently. AWS documents asynchronous completion through SNS [5], while S3 event notifications may be delivered more than once; idempotent processing is therefore an operational requirement rather than an optional optimisation [22].

IX. CONCLUSION AND FUTURE WORK

This work delivers an operational PDF-to-ranking pipeline in which asynchronous S3, Lambda, Textract, and SNS processing preserves extraction provenance from the first document event. Deterministic candidate chunks are embedded with voyage-4-large, indexed in MongoDB Atlas, retrieved through BM25 and vector search, fused through chunk-level RRF, and deduplicated into a focused candidate pool. Reconstructed resumes are then evaluated jointly with the job description by rerank-2.5, while retrieval ranks, matched chunks, and source references remain available for review.

The architecture addresses two recurring screening problems within one auditable workflow. BM25 retains exact technologies and role terminology, vector retrieval captures related meaning expressed with different wording, and RRF combines both rankings without forcing incompatible raw scores onto a common scale. Cross-encoder evaluation is concentrated on the retrieved pool, enabling richer job-resume comparison without applying pairwise processing to the entire corpus. The pilot demonstrates complementary candidate discovery and meaningful rank refinement while retaining a traceable connection to the indexed resume chunks.

Future implementation will extend this foundation with recruiter-labelled evaluation across multiple job families, requirement-to-resume evidence alignment, evidence-faithful explanations, and configurable human-review controls. Subsequent work will add alternative-opening matching and candidate communication based on the same reviewed evidence record, followed by systematic measurement of extraction quality, retrieval coverage, ranking agreement, latency, storage growth, and operational cost.

REFERENCES

- [1] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [3] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.
- [4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Recip-

- rocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd ACM SIGIR*, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.
- [5] Amazon Web Services, "Calling Amazon Textract asynchronous operations," *Amazon Textract Developer Guide*. [Online]. Available: <https://docs.aws.amazon.com/textract/latest/dg/api-async.html>. [Accessed: Oct. 1, 2026].
 - [6] E. Kesim and A. Deliahmetoglu, "Named entity recognition in resumes," arXiv:2306.13062, 2023.
 - [7] M. Zhang, K. Jensen, S. Sonniks, and B. Plank, "SkillSpan: Hard and soft skill extraction from English job postings," in *Proc. NAACL-HLT*, 2022, pp. 4962–4984.
 - [8] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
 - [9] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.
 - [10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.
 - [11] D. F. Mujtaba and N. R. Mahapatra, "Fairness in AI-driven recruitment: Challenges, metrics, methods, and future directions," arXiv:2405.19699, 2024.
 - [12] Voyage AI, "Text embeddings," 2026. [Online]. Available: <https://docs.voyageai.com/docs/embeddings>. [Accessed: Oct. 1, 2026].
 - [13] MongoDB, "MongoDB Vector Search overview," 2026. [Online]. Available: <https://www.mongodb.com/docs/vector-search/>. [Accessed: Oct. 1, 2026].
 - [14] MongoDB, "Return the score details: MongoDB Search," 2026. [Online]. Available: <https://www.mongodb.com/docs/search/query/score/get-details/>. [Accessed: Oct. 1, 2026].
 - [15] Voyage AI, "Rerankers," 2026. [Online]. Available: <https://docs.voyageai.com/docs/reranker>. [Accessed: Oct. 1, 2026].
 - [16] Government of India, *The Digital Personal Data Protection Act, 2023*. New Delhi, India: Ministry of Electronics and Information Technology, 2023.
 - [17] New York City Council, "Local Law 144 of 2021: Automated employment decision tools," New York City Administrative Code, enforcement effective Jul. 2023.
 - [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
 - [19] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.
 - [20] M. G. Kendall, "A new measure of rank correlation," *Biometrika*, vol. 30, no. 1/2, pp. 81–93, 1938.
 - [21] C. Spearman, "The proof and measurement of association between two things," *Amer. J. Psychol.*, vol. 15, no. 1, pp. 72–101, 1904.
 - [22] Amazon Web Services, "Amazon S3 event notifications," *Amazon S3 User Guide*, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html>. [Accessed: Oct. 1, 2026].
 - [23] Amazon Web Services, "Protecting data with server-side encryption using Amazon S3 managed encryption keys," *Amazon S3 User Guide*, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingServerSideEncryption.html>. [Accessed: Oct. 1, 2026].
 - [24] MongoDB, "MongoDB Search text operator," 2026. [Online]. Available: <https://www.mongodb.com/docs/atlas/atlas-search/text/>. [Accessed: Oct. 1, 2026].
 - [25] Voyage AI, "Flexible dimensions and quantization," 2026. [Online]. Available: <https://docs.voyageai.com/docs/flexible-dimensions-and-quantization>. [Accessed: Oct. 1, 2026].
 - [26] Voyage AI, "Reranker API reference," 2026. [Online]. Available: <https://docs.voyageai.com/reference/reranker-api>. [Accessed: Oct. 1, 2026].

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **23 Not Cited or Quoted 12%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 12%  Internet sources
- 2%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions

about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



0 AI-generated only 0%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



**CONFERENCE / JOURNAL PAPER PUBLICATION CURRENT
SUBMISSION DETAILS**

Manuscript ID	ICECONF2026-T1-1376
Manuscript type	Artificial Intelligence & Machine Learning
Title	A Hybrid Retrieval and Cross-Encoder Reranking Architecture for AI-Assisted Resume Screening and Talent Acquisition
Authors	DARSHAN M, DEEPA SURESHKUMAR, DANIEL LEVE MANICKAM D A, DAYANITHI V, JINU SOPHIA J
Date of submission	2026-10-03
Current status	Submitted

Manuscript Details

ICECONF2026-T1-1376

Manuscript Type:

Artificial Intelligence & Machine Learning

Title:

A Hybrid Retrieval and Cross-Encoder Reranking Architecture for AI-Assisted Resume Screening and Talent Acquisition

Abstract:

Large applicant collections require candidate generation to be efficient without reducing screening to exact keyword overlap. This paper describes an implemented retrieval-and-reranking pipeline for PDF resumes. An Amazon S3 event invokes asynchronous Amazon Textract processing, and the resulting text and layout blocks are retained as provenance-bearing JSON. A deterministic section-aware procedure divides each resume into candidate chunks. Voyage voyage-4-large produces 1024-dimensional representations, while MongoDB Atlas independently performs BM25 lexical retrieval and vector semantic retrieval. Reciprocal Rank Fusion combines the two chunk rankings and deduplicates them into a candidate pool. Candidate documents are reconstructed from stored chunks and jointly evaluated with the job description by the configured Voyage rerank-2.5 cross-encoder. Its output is treated as a semantic relevance score rather than a hiring probability. A repository-backed pilot illustrates complementary retrieval coverage and rank movement, but does not establish ranking accuracy. Requirement-to-resume evidence alignment, explainable recommendations, alternative-opening matching, and candidate communication are presented only as proposed extensions for human-reviewed decision support.

Keywords:

hybrid retrieval,BM25 lexical retrieval,vector semantic retrieval,Reciprocal Rank Fusion,candidate chunk,crossencoder reranking

Classification:

Artificial intelligence,Machine learning (general),Image processing,Cloud computing

Date of Submission:

2026-10-03

Authors

Author List:

DARSHAN M, Deepa Sureshkumar, Daniel Leve Manickam D A, Dayanithi V, Jinu Sophia J,

Uploaded Files

Manuscript:

iceconf2026-T1-1376_manuscript.pdf

Image:

Not Uploaded

Table:

Not Uploaded

Research Data:

Not Uploaded

Supplementary File:

Not Uploaded

Status

Current Status:

Submitted